Input: An array A of n number.

Output: Median of input

Naive Algo: Sort array A and output $\frac{n}{2}^{th}$ element.

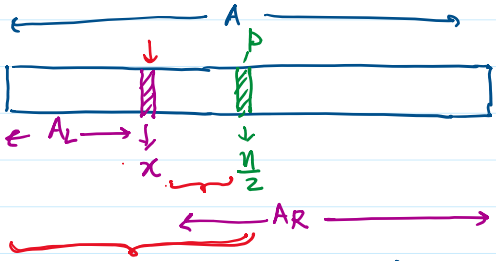
Time complexity: $O(n \log n)$

How to compute median in $O(n)$ time?

How to compute rank-r element in $O(n)$ time. $r \in \{1 \ldots n\}$

Def **Approximate Median** An element x is called approx median

$$\frac{n}{4} \leq \text{rank}(x) \leq \frac{3n}{4}$$



$p :=$ exact media

Case 1 $|A_L| < \frac{n}{2}$

$\Rightarrow$ median p will be in A_R

rank of p in A_R will be $\frac{n}{2} - |A_L| - 1$

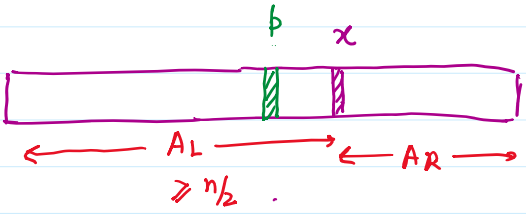
rank (x): position of x in sorted order in A

$\Rightarrow$ Search for an element of rank $\left(\frac{n}{2} - |A_L| - 1\right)$ in A_R

$\Rightarrow$ Search space is reduced by $\frac{n}{4}$

Case 2 $|A_L| > \frac{n}{2}$

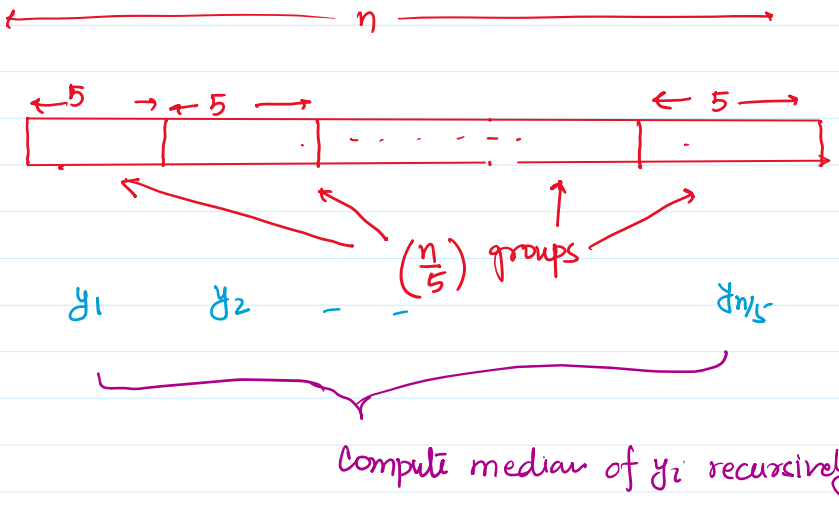
rank of p in A_L is $\frac{n}{2}$



$\Rightarrow$ Search for an element of rank $\frac{n}{2}$ in A_L

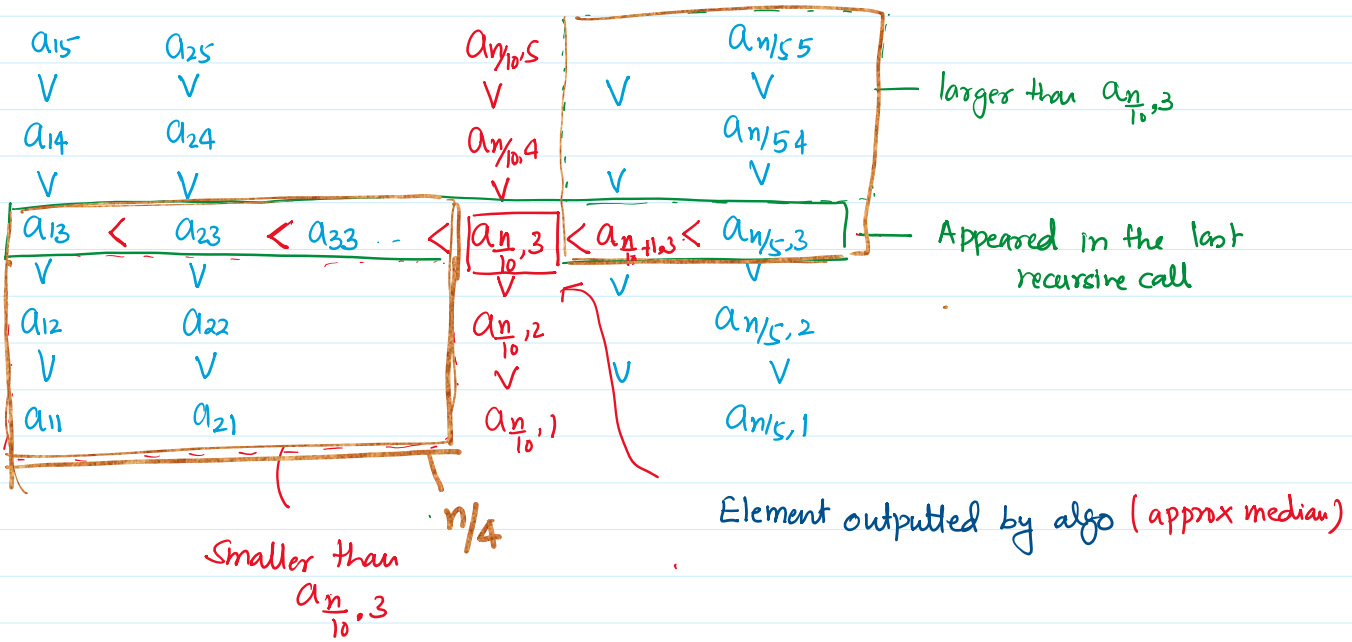
$\Rightarrow$ Search space is reduced by $\frac{n}{4}$

Que: How to compute approx median?



Approx-Median(A)

1. Divide the n elements into $\lfloor \frac{n}{5} \rfloor$ groups of 5 element each
2. Sort the elements of each group, and compute the median of each group (say $y_1, y_2 \cdots y_{n/5}$)
3. Recursively compute median of $y_1, y_2 \cdots y_{n/5}$



Thus we have shown that atleast $\frac{n}{4}$ elements are smaller than $a_{\frac{n}{10},3}$ and atleast $n/4$ elements are larger than $a_{\frac{n}{10},3}$. Therefore it is an approx-median

$$\frac{n}{4} < \text{rank}\left(a_{\frac{n}{10},3}\right) < \frac{3n}{4}$$

$\frac{4}{}$ $\frac{10}{}$ $\frac{}{4}$

FindElement(r, A) /* find an element of rank r in array A */

① Partition A into $\frac{n}{5}$ groups of size 5 each

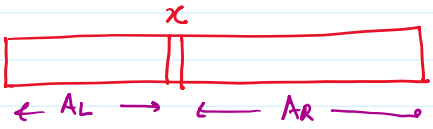
② Sort each 5 elements, compute median of each group, and put them in array B $|B| = \frac{n}{5}$

③ FindElement$\left(\frac{n}{10}, B\right)$

return element x. /* approx median */

Computing Approx Median

④ Partition A into A_L and A_R /* elements in A_L are smaller than x
" A_R " bigger " "



⑤ $|A_L| = r-1$ return x

⑥ If $|A_L| > r-1$, then FindElement(r, A_L)

⑦ If $|A_L| < r-1$, then FindElement$(r-|A_L|-1, A_R)$

/* Exact median */

$$T(n) = \underbrace{cn}_{\text{Step1+Step2}} + T\left(\underbrace{\frac{n}{5}}_{\text{Step3}}\right) + \underbrace{c'n}_{\text{Step4}} + T\left(\underbrace{\frac{3n}{4}}_{\text{Step 6 or 7 (as we are proceeding in either } A_L \text{ or } A_R)}\right)$$

$= O(n)$



FindMin

Input : Array A $|A| = n$
Output : Min element of array A

temp ← current min

A[i] < temp
temp ← A[i]

of comparision $n-1$

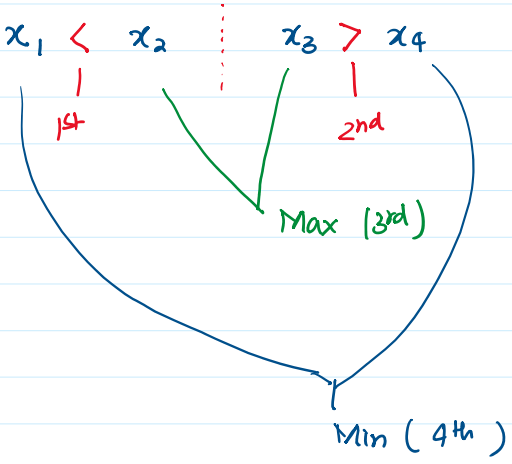
A[i] < temp
temp ← A[i]

loop over n element

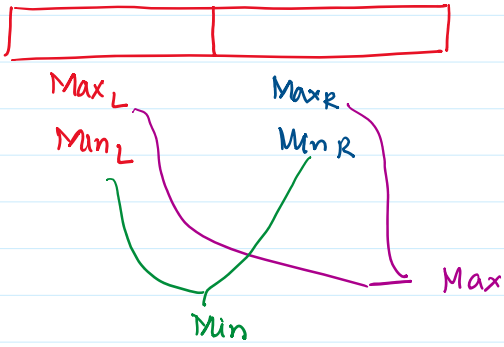
Problem: Max Min

Naive Algo 2(n-1) comparision

Que: Compute Max Min in lesser than 2(n-1) comparision



Naive algo require 6 comparision



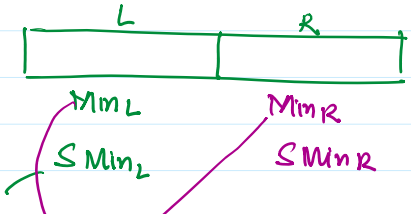
- Divide input into two halves (L, R)
- Recurse on both parts
- Compute the final soln by combining the soln of two parts

$T(n) = 2T(n/2) + 2$ $T(2) = 1$

$= \frac{3n}{2} - 2$

Min and 2nd Min

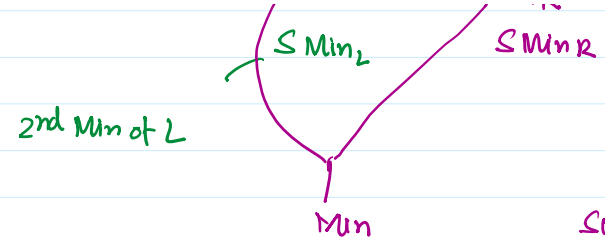
Naive algo: require 2(n-1) comparision



$T(n) = 2T(\frac{n}{2}) + 2$

$T(n) = 2T(\frac{n}{2}) + 2$

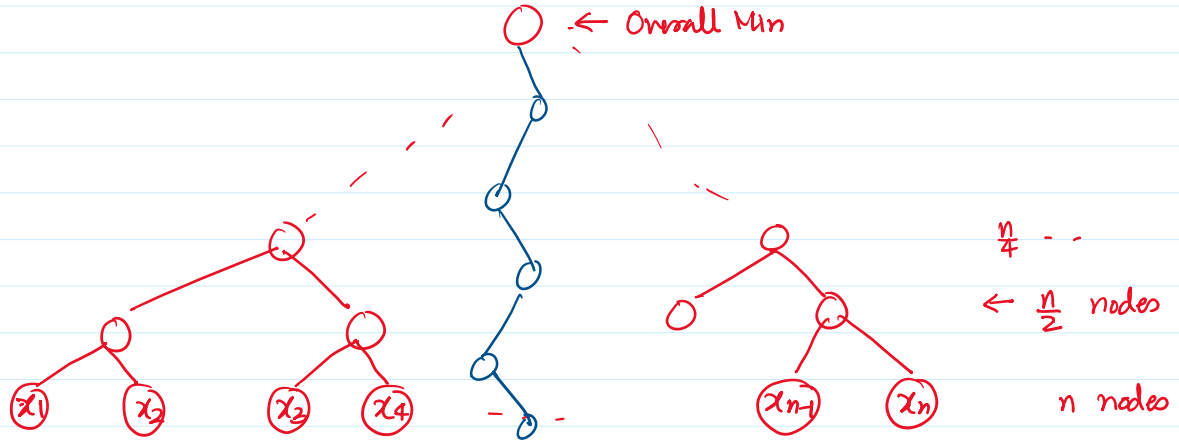
$= \frac{3n}{2} - 2$



Suppose $Min_L < Min_R$

Compare $SMin_L$ and Min_R

Ex: For Max Min Problem $\frac{3n}{2} - 2$ is the best



Observation At some stage Overall Min must have been compared with 2nd Min element.

Overall Min must have been compared with $\log n$ element in tree.
List down $\log n$ elements with which "overall Min" was compared

$(n-1)$ comparision + $(\log n - 1)$ comparision

↑ Overall Min ↑ 2nd Min